SASTI DAWAI

Backend Product Requirements Document

FastAPI • PostgreSQL • AI/ML orchestration

Version 1.0 • 13 October 2026
Aligned with the main Sasti Dawai PRD and ML PRD

SAFE BY DESIGN • EVIDENCE-BASED • MOBILE-FIRST

Purpose & boundaries

Sasti Dawai helps users scan Indian medicine packaging, verify the printed medicine details, compare against exact lower-cost Jan Aushadhi products where available, calculate savings from user-confirmed usage, and find a nearby Kendra. The backend is the trusted API, data, business-rules, and AI-orchestration layer.

REQUIRED BACKEND: Python + FastAPI. AI may read packaging and retrieve candidates; verified catalogue data and deterministic code decide medicine equivalence and savings.

Goals

- Expose documented, versioned APIs for scan, confirmation, matching, search, savings, monthly basket, Kendra discovery, feedback, and data provenance.
- Coordinate vision extraction, OCR/barcode adapters, resolver, confidence gate, and deterministic matcher.
- Validate all inputs/outputs with Pydantic; fail safely when data is missing, uncertain, stale, or contradictory.
- Support anonymous use, mobile networks, English/Hindi UI, useful errors and measurable latency.

Out of scope

- Diagnosis, dosing advice, prescription interpretation, treatment decisions, medicine sales or payments.
- LLM-generated equivalence, price, stock status, or invented catalogue rows.
- Raw-photo retention by default or unnecessary patient-identifying data.

Architecture & technology

Layer	Choice	Responsibility
API	FastAPI + Uvicorn	HTTP routes, OpenAPI, dependency injection, async request handling.
Schemas	Pydantic v2	Strict request/response validation.
Database	PostgreSQL	Products, ingredients, prices, equivalence rules, Kendras, feedback and provenance.
Search	pg_trgm; optional pgvector	Fuzzy text retrieval; vectors only retrieve candidates, never prove equivalence.
Persistence	SQLAlchemy 2.x + Alembic	Database models, queries and versioned migrations.
AI integration	Python provider adapters	Vision model, OCR, barcode and explainer integrations.
Tests	pytest + HTTPX	Unit, API, integration and safety tests.
Observability	Structured logs; optional Sentry/OpenTelemetry	Latency, failures, request IDs and model version without logging photos.

Request flow

```
React PWA → FastAPI route → Pydantic validation → scan orchestrator → quality gate / vision / OCR / barcode → brand resolver → evidence & confidence gate → deterministic matcher → price/savings service → JSON response
```

Use a modular monolith for the hackathon, not microservices. Keep route handlers thin, business logic in services, database operations in repositories, and AI calls behind replaceable provider interfaces.

Suggested repository structure

```
backend/app/main.py
backend/app/api/v1/routes/
backend/app/core/config.py
backend/app/db/{models,session}.py
backend/app/schemas/
backend/app/repositories/
backend/app/services/{scan_orchestrator,matching,savings,kendras,feedback}.py
backend/app/ml/{vision_provider,ocr,barcode,resolver,safety_gate}.py
backend/app/data/etl/
backend/tests/{unit,integration}/
backend/alembic/
backend/.env.example
backend/README.md
```

Database & data pipeline

Table	Purpose
salts	Canonical ingredient IDs, names, aliases, moiety, NTI flag and notes.
products	Brand/generic name, manufacturer, form, release type, route, pack size, MRP, schedule, source and price date.
product_salts	One row per active ingredient and strength; supports fixed-dose combinations.
equivalence	Reviewed brand/generic relation, tier, reason, reviewer and review date.
price_snapshots	Product price, source and snapshot date.
kendras	Address, PIN, district/state, coordinates, phone/hours if known, source and geocode date.
scans	Opaque scan ID, status, tier, latency and extractor version; no image/PII by default.
feedback	Scan ID, corrected field/value and timestamp; corrections remain untrusted until reviewed.
data_sources	Source URL/name, retrieval time, row count, checksum, licence notes and ETL version.

ETL requirements

- Download permitted official Jan Aushadhi/PMBI product and Kendra data; save raw files and record retrieval date/checksum.
- Parse into staging tables, normalize units/forms/release markers, and retain original source values.
- Represent every active ingredient separately. Never store a combination product only as an unparsed text string.
- Deduplicate using stable identifiers and product attributes; log ambiguous rows rather than silently dropping them.
- Store price date and source with every price; never present stale data as current.
- Do not invent missing prices, equivalence records, Kendra locations or stock status.

API contracts

Endpoint	Method	Purpose
/health	GET	Liveness check.
/ready	GET	Readiness check for database/configuration.
/v1/scan	POST	1–3 images; return extraction, field status, warnings, candidates and next action.
/v1/scan/{scan_id}/confirm	POST	Accept user-confirmed/corrected fields, revalidate and run matching.
/v1/match	POST	Match confirmed canonical fields against verified products.
/v1/search	GET	Search by brand/salt using exact and fuzzy lookup.
/v1/savings/estimate	POST	Savings from verified prices and user-confirmed units/month.
/v1/basket/summary	POST	Compute monthly/yearly basket totals; local-only basket is acceptable for MVP.
/v1/kendras/nearby	GET/POST	Nearby Kendras by coordinates or PIN/city; no claim of live stock.
/v1/feedback	POST	Record correction feedback without identity or image by default.
/v1/meta/data-sources	GET	Source names, last-updated timestamps and catalogue counts.

Example scan response (illustrative)

```
{
  "scan_id": "opaque-id",
  "status": "needs_confirmation",
  "extraction": {"brand_name": "Example Brand", "composition": [], "mrp_inr": null},
  "field_status": {"brand_name": "high", "composition": "medium", "release_type": "low"},
  "candidate_products": [],
  "warnings": ["Release type could not be verified"],
  "next_action": "confirm_or_retake",
  "model_version": "configured-version",
  "latency_ms": 1420
}
```

- Keep response contracts in Pydantic and synchronize TypeScript types from OpenAPI where practical.
- Validate file type, size and count. Use clear errors; never return success with a fabricated match.
- Use request IDs, stable error codes, pagination for large searches, and versioned endpoints.
- For invalid requests use suitable 4xx responses; provider/database failures must be recoverable and must not imply a match.

Matching, savings & medical safety

Only deterministic backend code may issue EXACT, CLOSE or NONE. EXACT requires the same complete active-ingredient set, strength per unit, dosage form, release type and route, with critical fields verified.

- Every ingredient and strength must match for combination products.
- SR/ER/XR/MR products are not exact matches for immediate-release products.
- Different strength, unknown release type, partial ingredient overlap, unresolved salt form, or evidence contradiction cannot be EXACT.
- Fuzzy/vector scores retrieve candidates only and cannot override deterministic rules.
- Low confidence or OCR/VLM/catalogue disagreement requires confirmation or NONE.
- NTI medicines use a reviewed list and prominent warning; suppress celebratory savings emphasis.

Savings formula

```
brand_unit_price = brand_mrp / brand_pack_count
generic_unit_price = generic_mrp / generic_pack_count
monthly_units = user_confirmed_units_per_day * 30
monthly_saving = (brand_unit_price - generic_unit_price) * monthly_units
yearly_saving = monthly_saving * 12
```

Only verified EXACT matches count toward savings. Compare like units (tablet to tablet, ml to ml). Never infer dose frequency from packaging.

Every result must include the fixed sentence: “Confirm with your doctor or pharmacist.” The backend should return a fixed disclaimer field; the frontend must keep it visible.

AI/ML integration

- Use a vision-provider interface so the configured vision model can be changed without rewriting routes.
- Version prompts and schemas. Require structured JSON output and validate every response with Pydantic.
- Treat OCR, barcode, VLM and catalogue values as evidence with provenance, not unquestionable truth.
- Resolver sequence: exact lookup → lexical fuzzy search (pg_trgm/RapidFuzz) → optional vector search (pgvector) → evidence-aware reranking.
- Confidence/safety gate checks critical fields, evidence conflicts and user confirmation before matching.
- Explainer LLM may rephrase verified facts only. Prices, tier, savings, warnings and disclaimer are controlled by code.
- Record model/provider and prompt versions, latency, outcome and safe error codes. Do not log images by default.

If an AI provider fails, do not guess. Offer retry, manual search, or a deterministic template explanation.

Security & non-functional requirements

- Parent PRD scan latency target: p50 ≤ 6 s and p95 ≤ 10 s on 4G; measure it.
- Store keys in deployment secrets/environment variables; never commit .env.
- Use HTTPS, CORS allowlists, request size limits and rate limits on expensive scan endpoints.
- Use parameterized ORM queries, least-privilege DB credentials and versioned migrations.
- No login required for MVP; use opaque scan IDs if tracking is needed.
- Do not persist photos by default. Avoid logging personal details, raw image payloads or unnecessary location data.
- Expose health/readiness checks and safe public errors; never leak stack traces or secrets.

Testing & acceptance

Required tests

- Unit tests for unit conversion, ingredient-set comparison, combinations, release type, route, form and savings arithmetic.
- API tests for invalid images, malformed model JSON, timeouts, no candidate, database failure and conflicting evidence.
- Safety tests proving fuzzy/vector similarity alone can never produce EXACT.
- ETL tests for duplicates, missing fields, malformed prices, combinations and source metadata.
- Contract tests ensure responses match frontend TypeScript types.

Definition of done

- Frontend can upload an image and receive a typed FastAPI response.
- Uncertain/conflicting critical fields require confirmation.
- Zero false EXACT results on the golden test set; any failure blocks release.
- Prices show freshness/source; only EXACT results count toward savings.
- All critical endpoints have tests, documented errors and OpenAPI descriptions.
- README, .env.example, migrations and reproducible ETL are provided.
- Latency and failures can be inspected without exposing images or personal data.

Hackathon build order

- Build schema, Pydantic contracts, /health and seeded catalogue first.
- Implement normalization, deterministic matcher and savings tests before vision integration.
- Add scan orchestration, provider adapter, schema validation and safe error states.
- Add confirmation, fuzzy search, Kendra endpoint and feedback.
- Integrate frontend early and test on a real phone over mobile data.
- Run the golden-set evaluation, fix high-risk errors, freeze features and rehearse failure paths.

Avoid microservices, complex queues, authentication and vector search until scan → confirm → exact match → savings works reliably.

Glossary

Term	Simple meaning
FastAPI	Python framework for building web APIs.
Endpoint	A URL and HTTP method that performs one operation.
Pydantic	Validates the shape and types of Python data.
SQLAlchemy / ORM	Lets Python interact with database tables through models and queries.
Alembic migration	A repeatable, versioned database structure change.
ETL	Extract, Transform, Load: collect, clean and store source data.
CORS	Browser rule controlling which website can call the API.
Rate limit	Limit on how often a client can call an endpoint.
pg_trgm	PostgreSQL approximate text matching extension.
pgvector	PostgreSQL extension for vector/embedding search.
Fail closed	Refuse a risky action when uncertain rather than guess.
Provenance	Where data came from and when it was retrieved.
p50/p95	Median latency / latency met by 95% of requests.

AI coding-agent instructions

- Inspect the repository and actual datasets first. Preserve working code and report discrepancies.
- Implement FastAPI + Pydantic v2 + SQLAlchemy + PostgreSQL. Keep routes thin and separate services, repositories and AI adapters.
- Do not fabricate medicine data, prices, Kendra locations, credentials, model outputs or evaluation metrics.
- Write safety tests before changing exact-match behavior; keep all equivalence rules deterministic.
- Provide .env.example, migrations, ETL scripts, tests, OpenAPI docs and README with run/deploy commands.
- Report actual tests, completed endpoints, known limitations and missing credentials/data. Never claim unverified deployment success.